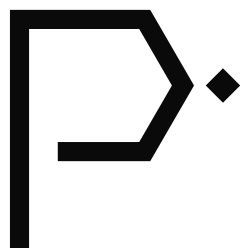




SENTINELLE
SERVICES



POSTAGENT

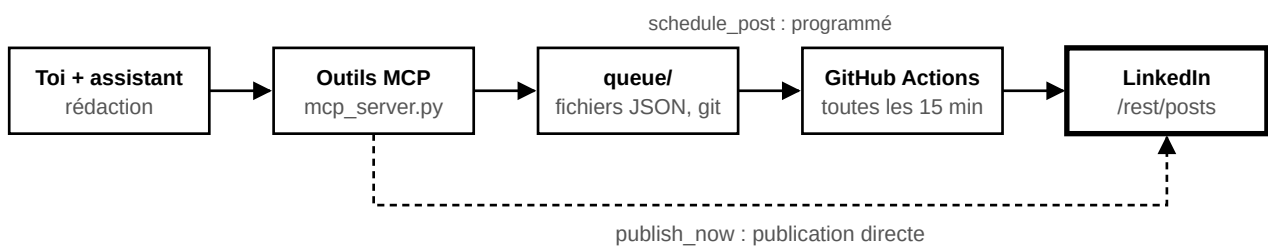
GUIDE D'UTILISATION

Publier sur LinkedIn depuis une conversation, avec une planification qui tourne sans serveur.

Guide d'utilisation au quotidien. Pour le *pourquoi* des choix, voir ARCHITECTURE.md ; pour les tests, TESTING.md .

Le principe en trois lignes

1. Tu rédiges le post **dans la conversation** avec l'assistant. Le code ne rédige rien, il reçoit un texte final.
2. L'assistant publie tout de suite, ou programme le post en écrivant un fichier dans `queue/` qu'il pousse sur GitHub.
3. GitHub Actions, réveillé toutes les 15 minutes par l'horloge Cloudflare, publie les posts dont l'heure est venue — poste éteint ou non.



1. Mise en route (une seule fois)

◆ Installation locale

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt          # publication
pip install -r requirements-mcp.txt     # serveur MCP (SDK épinglé à 2.2.0)
cp .env.example .env                   # puis remplir Client ID et Client Secret
```

Lance les deux `pip install` **l'un après l'autre**, jamais en parallèle : deux installations concurrentes corrompent le `site-packages` (constaté le 23/09/2026).

◆ Prérequis côté poste

- `gh` connecté au compte qui possède le dépôt (`gh auth status`) : il sert à mettre à jour le secret GitHub du token.
- Une clé GPG utilisable sans saisie interactive (agent `gpg` déverrouillé) : les commits faits depuis la conversation sont signés, et une demande de phrase de passe invisible bloquerait l'outil 30 secondes avant d'échouer.

◆ Authentification LinkedIn

```
python auth.py
```

Le navigateur s'ouvre, tu autorises l'app `Post-Agent` . Le script, d'un seul geste :

- écrit `token.json` (token + URN, jamais commité) ;
- met à jour le secret GitHub `LINKEDIN_ACCESS_TOKEN` ;
- committe et pousse `token_expiry.json` (la date seule, pour l'alerte).

Si seul le dernier push échoue, le script le dit et donne la commande à rejouer. **Ne relance pas** `auth.py` **dans ce cas** : le token est bon.

◆ Secrets GitHub du dépôt

SECRET	RÔLE	POSÉ PAR
LINKEDIN_ACCESS_TOKEN	publier en CI	auth.py , automatiquement
LINKEDIN_PERSON_URN	auteur des posts	à la main, une fois
CLOUDFLARE_API_TOKEN , CLOUDFLARE_ACCOUNT_ID	déployer l'horloge	à la main, une fois
DISPATCH_PAT	l'horloge déclenche le workflow	à la main, expire au bout d'un an

Détail de l'horloge : clock/README.md .

◆ Brancher l'assistant

Le fichier .mcp.json déclare le serveur post-agent (lancé par .venv/bin/python mcp_server.py).
Ouvre Claude Code à la racine du dépôt : les cinq outils apparaissent sous mcp__post-agent__* . Vérifie avec :

« Quel est l'état de mon token LinkedIn ? »

2. Utilisation courante, par la conversation

Tu parles normalement ; l'assistant choisit l'outil. Voici ce que fait chacun, et ce qu'il faut savoir.

◆ Rédiger

Rien de spécial : demande un brouillon, corrige-le, itère. Quand le texte te convient, dis-le explicitement et précise **la visibilité**.

Points de rédaction que le système impose :

- **3 000 unités maximum**, comptées en UTF-16 sur le texte échappé. Un emoji compte double, et les caractères `( ) [ ] { } @ * _ ~ | < >` prennent un caractère de plus chacun. Le refus arrive avant tout appel réseau, avec le compte exact.
- Les hashtags `#` restent actifs, le reste s'affiche tel quel (vérifié).
- Un post publié **ne peut pas être modifié** par l'API : corriger veut dire supprimer et republier, en perdant les réactions. Relis avant.

◆ Visibilité : toujours explicite

`PUBLIC` ou `CONNECTIONS`. Il n'y a pas de valeur par défaut, nulle part. Si tu ne la dis pas, l'assistant doit te la demander. C'est voulu : un défaut « raisonnable » est la cause directe de l'incident du 14/09/2026.

◆ Publier maintenant — `publish_now`

« Publie ce texte maintenant, en `PUBLIC`. »

Retour : le `post_id`, le chemin du journal dans `published/`, et la commande de suppression. Garde ce `post_id` : l'API ne permet pas de retrouver ses propres posts, c'est lui seul qui permet de supprimer.

Si le retour contient un `warning` : **le post est en ligne**, seul le journal n'a pas été poussé. Ne demande surtout pas de relancer — ce serait un doublon public. Pousse le journal à la main (`git push`).

◆ Programmer — `schedule_post`

« Programme ce post pour demain à 9 h, en `CONNECTIONS`. »

- L'heure part avec son fuseau (`2026-10-03T09:00:00+01:00`). Sans fuseau, l'outil refuse plutôt que de deviner.
- Une heure passée est refusée ; deux posts à la même minute aussi.
- Le post n'est programmé **que si le retour indique** `pushed: true`. Un fichier écrit mais non poussé n'existe pas pour GitHub Actions.

Le fichier créé s'appelle `queue/AAAA-MM-JJTHMM.json`, à l'heure locale du poste. C'est ce nom qui sert à l'annuler.

◆ Consulter la file — `list_queue`

« Qu'est-ce qui est programmé ? »

Quatre listes, avec un extrait de 80 caractères par post :

LISTE	SIGNIFICATION	ACTION
<code>queue</code>	programmés, en attente	rien
<code>publishing</code>	publication interrompue ou fichier invalide	à toi de trancher , voir §4
<code>stale</code>	trop en retard, jamais publiés	à toi de trancher , voir §4
<code>published_recent</code>	5 derniers publiés, avec <code>post_id</code>	rien

◆ Annuler — `cancel_post`

« Annule le post de demain 9 h. »

L'assistant passe le nom de fichier exact (consulte la file d'abord si besoin). Le fichier est supprimé et le retrait poussé ; git garde l'historique. Refusé si le post est déjà en `publishing/` ou publié : trop tard, vérifie le profil.

◆ Modifier un post programmé

Pas d'outil dédié : **annule, puis reprogramme** le texte corrigé. Deux commits, aucune ambiguïté.

◆ État du token — `auth_status`

« Mon token est-il encore valide ? »

Donne les jours restants, `alerte: true` sous 7 jours, et `secret_in_sync` :

- `true` : le secret GitHub est à jour ;
- `false` : la CI utilisera un vieux token → relance `python auth.py` ;
- `null` : GitHub injoignable, état **inconnu** (pas une divergence).

3. Ce qui se passe sans toi

Toutes les 15 minutes, le Worker Cloudflare déclenche le workflow `Publication LinkedIn`. Le cron GitHub tourne aussi, en secours.

À chaque run, `publisher.py --commit` :

1. range les fichiers invalides dans `publishing/` et les posts dus depuis plus de **3 h** dans `stale/` (sans les publier) ;
2. pour chaque post dû dans les **20 prochaines minutes**, attend l'heure exacte, le déplace dans `publishing/` et **pousse** ce verrou ;
3. publie, écrit le `post_id` dans `published/`, pousse.

Précision mesurée : 1,7 s de dérive. Deux runs ne traitent jamais la file en même temps (`concurrency: publisher`).

Conséquence pratique : **fais un `git pull` avant de toucher la file à la main**, la CI commite dans le dépôt à chaque publication. Les outils MCP le font d'eux-mêmes.

4. Quand quelque chose ne va pas

GitHub t'envoie un e-mail à chaque run en échec. Voici comment lire la situation.

◆ Un fichier dans `publishing/`

Deux cas, que le journal du run distingue :

- « **Fichier de queue invalide, mis de côté** » : JSON cassé, fuseau absent, visibilité manquante. Rien n'est parti. Corrige le fichier, remets-le dans `queue/` , pousse.
- « **laissé dans publishing/, arbitrage manuel** » : l'appel LinkedIn a échoué... ou a réussi sans que la réponse revienne. **Regarde ton profil avant tout.**
 - Le post est en ligne → déplace le fichier dans `published/` (sans `post_id` , note-le en commentaire du commit) ;
 - il n'y est pas → corrige la cause (token ?), remets le fichier dans `queue/` .

Un fichier ne repart jamais tout seul de `publishing/` . C'est volontaire : mieux vaut un post bloqué qu'un doublon.

◆ Un fichier dans `stale/`

Le post a raté son créneau de plus de 3 h — en général l'horloge est tombée et seul le cron GitHub, très irrégulier, a tourné. Vérifie les journaux du Worker (dashboard Cloudflare), puis décide : abandonner, ou **redater** `scheduled_at` (et le nom du fichier) et le remettre dans `queue/` .

◆ Token qui expire

- À moins de 7 jours, le workflow ouvre une issue `token-expiry` (un seul e-mail).
- Relance `python auth.py` : token local, secret GitHub et date d'expiration sont mis à jour ensemble. Ferme l'issue ensuite.
- Le token actuel expire le **13/11/2026**. Pas de renouvellement automatique possible avec cette app LinkedIn.

◆ Erreurs fréquentes

SYMPTÔME	CAUSE PROBABLE	QUE FAIRE
outil bloqué ~30 s puis « n'a pas rendu la main »	GPG attend une phrase de passe	déverrouille l'agent gpg, relance
401 en CI	token expiré ou secret divergent	<code>auth_status</code> , puis <code>python auth.py</code>
426	version d'API <code>LinkedIn-Version</code> périmée	mettre à jour (<code>202608</code> à remonter avant l'été 2027)
429	quota journalier	attendre, ne pas réessayer en boucle
conflit de rebase	file modifiée des deux côtés	résoudre à la main ; rien n'a été publié

Tableau complet : `TESTING.md` , section « Tableau des erreurs connues ».

5. Depuis le terminal (secours)

Quand l'assistant n'est pas disponible :

```
python auth.py --status           # état du token et du secret CI
python publisher.py --days-left   # jours restants
python publisher.py --publish-now "texte" --visibility CONNECTIONS
python linkedin.py --me           # identité, sans rien publier
python linkedin.py --delete urn:li:share:... # supprimer un post publié
gh workflow run publish.yml       # forcer un run du publisher
```

Pour programmer à la main, écris un fichier dans `queue/` :

```
{
  "text": "Le contenu du post.",
  "scheduled_at": "2026-10-03T09:00:00+01:00",
  "visibility": "CONNECTIONS",
  "created_at": "2026-10-02T22:00:00+01:00"
}
```

puis `git add queue/ && git commit && git push` . Non poussé, il n'existe pas.

N'utilise jamais `linkedin.publish()` directement : il ne journalise rien, et un post sans `post_id` noté ne se supprime plus que depuis l'interface web.

6. Règles à ne jamais oublier

- **Visibilité toujours explicite.** Aucun défaut.
- **Garde chaque** `post_id` . Sans lui, pas de suppression par l'API.
- **Après un échec, regarde le profil avant de relancer.** Une erreur HTTP ne prouve pas que rien n'est parti.
- **Pour tester,** `POSTAGENT_DRY_RUN=1` . LinkedIn n'a pas de bac à sable : tout appel réussi est une vraie publication. Un post de test est en `CONNECTIONS` et supprimé dans la foulée.
- **Deux échéances à surveiller** : le token LinkedIn (2 mois) et `DISPATCH_PAT` (1 an).

Ce que le système ne fait pas

Images (prévu, étape 5), statistiques, modification du profil, publication sur X, interface web. Pas de relecture des posts publiés : l'API la refuse avec les droits accordés.